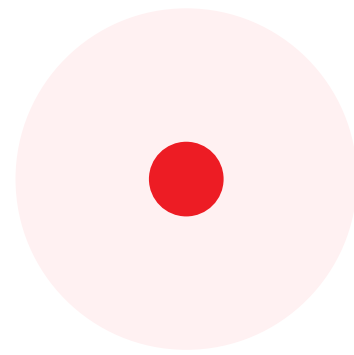




Graduate & Junior

Developer Assignment

Engineering Foundations: from a small calculation program to a tested, documented, deployable service.

01

Fibonacci

02

Factorial

03

Loan repayment

What this tests

Correctness, clean code, testing discipline, pragmatic design, documentation, and the ability to evolve a simple program into useful software.

Problem description

Build a small application that performs three calculations, then evolve it into a simple, production-minded service. Candidates may use any mainstream language or framework, ideally Python

1 Fibonacci

Input: n

Output: nth Fibonacci number

- Handle $n = 0$ and $n = 1$.
- Prefer an iterative or efficient approach.
- Reject invalid input.

2 Factorial

Input: n

Output: $n!$

- Handle $0! = 1$.
- Reject negative input.
- Consider large numbers responsibly.

3 Loan repayment

Inputs: principal, annual rate, months

Output: monthly repayment

- Use decimal-safe handling.
- Handle zero-interest loans.
- Round money clearly.

Loan formula

$$M = P \times \left[\frac{r(1+r)^n}{(1+r)^n - 1} \right]$$

P = principal, r = monthly interest rate, n = number of months, M = monthly repayment.

Then, you can extend the program as much as you want

Quality

- Separate business logic from input/output.
- Validate invalid inputs and edge cases.
- Keep design simple and readable.

Tests

- Cover normal cases and invalid inputs.
- Include at least one test per calculation.
- Show how to run the test suite.

API or UI

- Expose a REST API or simple web UI.
- Make inputs clear and errors understandable.
- Keep endpoints/screens minimal.

Delivery

- Add a README.md.
- Add Docker support if possible.
- Include assumptions and limitations.

Guidance and evaluation

This assignment is intentionally small. The engineering signal comes from how the candidate structures, tests, documents, and evolves the solution - not from over-engineering it.

Suggested implementation path

- 1 Core logic first**
Implement the three calculations in small, testable functions.
- 2 Validation next**
Reject impossible inputs and make error behavior predictable.
- 3 Tests before polish**
Add automated tests for normal, edge, and invalid cases.
- 4 Expose it**
Turn the logic into a CLI, API, or web UI with clear examples.
- 5 Package it**
Add README, Docker or another deployment instructions, and a short limitations section.

Evaluation criteria

Correctness Accurate calculations and sensible rounding.	Code clarity Readable structure and clear responsibilities.	Tests Useful coverage of realistic and edge cases.
Pragmatism Simple choices; no unnecessary complexity.	Documentation README that lets another engineer run it.	Delivery mindset Runnable, packaged, and easy to review.

Rationale

Fibonacci and factorial reveal basic programming discipline. Loan repayment adds financial relevance: decimals, rounding, validation, and edge cases. Together they create a fair junior-level exercise that can grow into a practical service.